



BIRZEIT UNIVERSITY

Faculty of Engineering and Technology

Department of Electrical and Computer Engineering

First semester 2025/2026

Applied Cryptography - ENCS4320

HW#2

Prepared by:

Nagham Massis 1220149

Al-zahra Nassif 1220168

Instructor: Dr. Ahmed Shawahna

Section: 2

Date: 27/1/2026

1. Phase I: Establishing Encrypted Communication

In this phase, a secure communication mechanism was established using symmetric cryptography. The goal was to ensure message confidentiality by implementing AES-128 encryption in CBC mode with PKCS#7 padding. A personal encryption key was derived from my student ID, and a fresh random initialization vector (IV) was generated for each encrypted message. The implementation supports both encryption and decryption and forms the basis for secure message exchange in later phases.

1.1 Personal Key Generation

The AES-128 encryption key was generated from my student ID using the MD5 hash function, which produces a 16-byte key suitable for AES. A reference IV was also derived from the SHA-256 hash of my student ID for testing purposes, while a new random IV is generated for every encrypted message during actual communication.

```
# =====  
# derive AES key and reference IV from my id  
# =====  
  
student_id = "1220168"  
  
# AES-128 key derived using MD5  
aes_key = hashlib.md5(student_id.encode("utf-8")).digest()  
  
# reference IV derived from SHA256 |  
reference_iv = hashlib.sha256(student_id.encode("utf-8")).digest()[:16]
```

1.2 Supporting Helper Functions

xor_bytes(a, b)

This function performs a byte-wise XOR operation between two byte arrays. XOR is used in CBC mode and in the AddRoundKey step of AES.

```
# =====  
# basic helper functions for CBC and AES  
# =====  
  
BLOCK_SIZE = 16 # AES block size is 16 bytes  
  
def xor_bytes(a, b):  
    # XOR two byte arrays of equal length  
    result = b""  
    for i in range(len(a)):  
        result += bytes([a[i] ^ b[i]])  
    return result
```

✚ pkcs7_pad(data)

This function applies PKCS#7 padding to the plaintext so that its length becomes a multiple of the AES block size (16 bytes).

```
def pkcs7_pad(data):  
    # add PKCS#7 padding to make length multiple of 16  
    pad_len = BLOCK_SIZE - (len(data) % BLOCK_SIZE)  
    padding = bytes([pad_len]) * pad_len  
    return data + padding
```

✚ pkcs7_unpad(data)

This function removes the PKCS#7 padding after decryption to recover the original plaintext.

```
def pkcs7_unpad(data):  
    # remove PKCS#7 padding  
    pad_len = data[-1]  
    return data[:-pad_len]
```

✚ random_iv()

This function generates a fresh random initialization vector (IV) for each encrypted message. The IV is not secret and is sent with the ciphertext.

```
def random_iv():  
    # generate random IV  
    return os.urandom(BLOCK_SIZE)
```

✚ split_blocks(data)

This function splits the padded data into fixed-size blocks of 16 bytes, which is required for AES block encryption.

```
def split_blocks(data):  
    # split padded data into 16-byte blocks  
    blocks = []  
    for i in range(0, len(data), BLOCK_SIZE):  
        blocks.append(data[i:i + BLOCK_SIZE])  
    return blocks
```

1.3 AES-128 Encryption Implementation

✚ bytes_to_state(block16)

This function converts a 16-byte block into a 4×4 AES state matrix using column-wise ordering.

```
def bytes_to_state(block16):  
    # convert 16 bytes into 4x4 AES state  
    state = [[0, 0, 0, 0] for _ in range(4)]  
    index = 0  
    for col in range(4):  
        for row in range(4):  
            state[row][col] = block16[index]  
            index += 1  
    return state
```

✚ state_to_bytes(state)

This function converts the AES state matrix back into a 16-byte sequence after encryption or decryption.

```
def state_to_bytes(state):  
    # convert AES state back to 16 bytes  
    out = []  
    for col in range(4):  
        for row in range(4):  
            out.append(state[row][col])  
    return bytes(out)
```

✚ sub_bytes(state)

This function applies the AES S-box substitution to each byte of the state to introduce non-linearity

```
def sub_bytes(state):  
    # apply S-box substitution to each byte in the state  
    for r in range(4):  
        for c in range(4):  
            state[r][c] = SBOX[state[r][c]]  
    return state
```

✚ shift_rows(state)

This function shifts the rows of the AES state to the left by different offsets, helping to spread byte dependencies.

```
# =====  
#                               AES shiftRows  
# =====  
def shift_rows(state):  
    # row 0: no shift  
    state[0] = state[0]  
  
    # row 1: shift left by 1  
    state[1] = state[1][1:] + state[1][:1]  
  
    # row 2: shift left by 2  
    state[2] = state[2][2:] + state[2][:2]  
  
    # row 3: shift left by 3  
    state[3] = state[3][3:] + state[3][:3]  
  
    return state
```

✚ mix_columns(state)

This function mixes each column of the AES state using finite field arithmetic to provide diffusion.

```
# =====  
#                               AES mix column  
# =====  
def xtime(a):  
    # multiply by 2 in GF(2^8)  
    a = a & 0xFF  
    a2 = a << 1  
    if a & 0x80:      # if highest bit was 1  
        a2 = a2 ^ 0x1B # reduce with AES polynomial  
    return a2 & 0xFF  
  
def mul_by_2(a):  
    return xtime(a)  
  
def mul_by_3(a):  
    # 3*a = (2*a) XOR a  
    return xtime(a) ^ a  
  
def mix_single_column(col):  
    # col is a list of 4 bytes: [a0,a1,a2,a3]  
    a0, a1, a2, a3 = col[0], col[1], col[2], col[3]  
  
    b0 = mul_by_2(a0) ^ mul_by_3(a1) ^ a2 ^ a3  
    b1 = a0 ^ mul_by_2(a1) ^ mul_by_3(a2) ^ a3  
    b2 = a0 ^ a1 ^ mul_by_2(a2) ^ mul_by_3(a3)  
    b3 = mul_by_3(a0) ^ a1 ^ a2 ^ mul_by_2(a3)  
  
    return [b0 & 0xFF, b1 & 0xFF, b2 & 0xFF, b3 & 0xFF]
```

```
def mix_columns(state):
    # apply MixColumns on each column of the state
    for c in range(4):
        col = [state[0][c], state[1][c], state[2][c], state[3][c]]
        mixed = mix_single_column(col)
        state[0][c] = mixed[0]
        state[1][c] = mixed[1]
        state[2][c] = mixed[2]
        state[3][c] = mixed[3]
    return state
```

✚ add_round_key(state, round_key16)

This function XORs the AES state with the round key. It is the step where the encryption key directly affects the data.

```
# =====
#                               AES AddRoundKey
# =====
def add_round_key(state, round_key16):
    # round_key16 is 16 bytes (one round key)

    idx = 0
    for col in range(4):
        for row in range(4):
            state[row][col] = state[row][col] ^ round_key16[idx]
            idx += 1

    return state
```

1.4 Key Expansion (AES Key Schedule)

✚ rot_word(word4)

This function rotates a 4-byte word to the left by one byte as part of the AES key expansion process.

```
def rot_word(word4):
    # rotate left: [a0,a1,a2,a3] -> [a1,a2,a3,a0]
    return word4[1:] + word4[:1]
```

✚ sub_word(word4)

This function applies the AES S-box to each byte of a key word during key expansion.

```
def sub_word(word4):
    # apply S-box to each byte in the word
    return [SBOX[b] for b in word4]
```

✚ xor_words(w1, w2)

```
def xor_words(w1, w2):  
    # XOR two 4-byte words  
    return [w1[i] ^ w2[i] for i in range(4)]
```

✚ key_expansion(key16)

This function generates all key words required for AES-128 from the original 16-byte key.

```
def key_expansion(key16):  
    # key16 is original AES key (16 bytes)  
    # the output result is list of 44 words, each word is 4 bytes  
    words = []  
  
    #1) split original key into 4 words (w0 to w3)  
    for i in range(4):  
        word = [  
            key16[4*i],  
            key16[4*i + 1],  
            key16[4*i + 2],  
            key16[4*i + 3]  
        ]  
        words.append(word)  
  
    #2) generate remaining words (w4 to w43)  
    i = 4  
    while i < 44:  
        temp = words[i - 1].copy()  
  
        # apply g func  
        if i % 4 == 0:  
            temp = rot_word(temp)          # RotWord  
            temp = sub_word(temp)          # SubWord  
            temp = xor_words(temp, RCON[i//4 - 1]) # XOR with Round coefficient  
  
        # generate new word  
        new_word = xor_words(words[i - 4], temp)  
        words.append(new_word)  
  
        i += 1  
    return words
```

✚ words_to_round_keys(words)

This function groups the generated key words into 11 round keys, each consisting of 16 bytes.

```

# =====
#           build round keys (11 keys)
# =====

def words_to_round_keys(words):

    round_keys = []

    # each round key = 4 words = 16 bytes
    # round 0 key = w0..w3
    # round 1 key = w4..w7
    # ...
    # round 10 key = w40..w43
    for r in range(11):
        start = r * 4
        rk_bytes = []

        # collect 4 words
        for w in range(start, start + 4):
            rk_bytes += words[w] # add the 4 bytes of the word

        round_keys.append(bytes(rk_bytes)) # 16 bytes

    return round_keys

```

1.5 AES-128 Decryption Implementation

inv_sub_bytes(state)

This function performs the inverse of SubBytes by replacing each byte using the inverse S-box (INV_SBOX).

```

def inv_sub_bytes(state):
    # inverse substitution using INV_SBOX
    for r in range(4):
        for c in range(4):
            state[r][c] = INV_SBOX[state[r][c]]
    return state

```

inv_shift_rows(state)

This function reverses the ShiftRows step by shifting the AES state rows to the right instead of the left.

```

def inv_shift_rows(state):
    # inverse of ShiftRows (shift right instead of left)
    state[0] = state[0] # no shift
    state[1] = state[1][-1:] + state[1][: -1] # right by 1
    state[2] = state[2][-2:] + state[2][: -2] # right by 2
    state[3] = state[3][-3:] + state[3][: -3] # right by 3
    return state

```


Inverse MixColumns

In this part, the inverse MixColumns operation was implemented to reverse the column mixing performed during AES encryption. The multiplication in the finite field $GF(2^8)$ was handled using `gf_mul(a, b)` function, and the inverse transformation was applied first on a single column and then on the entire AES state.

```
def gf_mul(a, b):
    a = a & 0xFF
    b = b & 0xFF
    res = 0
    for _ in range(8):
        if b & 1:
            res ^= a
        hi = a & 0x80
        a = (a << 1) & 0xFF
        if hi:
            a ^= 0x1B
        b >>= 1
    return res & 0xFF

def inv_mix_single_column(col):
    # inverse MixColumns uses constants: 0e 0b 0d 09
    a0, a1, a2, a3 = col[0], col[1], col[2], col[3]

    b0 = gf_mul(a0, 14) ^ gf_mul(a1, 11) ^ gf_mul(a2, 13) ^ gf_mul(a3, 9)
    b1 = gf_mul(a0, 9) ^ gf_mul(a1, 14) ^ gf_mul(a2, 11) ^ gf_mul(a3, 13)
    b2 = gf_mul(a0, 13) ^ gf_mul(a1, 9) ^ gf_mul(a2, 14) ^ gf_mul(a3, 11)
    b3 = gf_mul(a0, 11) ^ gf_mul(a1, 13) ^ gf_mul(a2, 9) ^ gf_mul(a3, 14)
    return [b0 & 0xFF, b1 & 0xFF, b2 & 0xFF, b3 & 0xFF]

def inv_mix_columns(state):
    # apply inverse MixColumns on each column of the state
    for c in range(4):
        col = [state[0][c], state[1][c], state[2][c], state[3][c]]
        mixed = inv_mix_single_column(col)
        state[0][c] = mixed[0]
        state[1][c] = mixed[1]
        state[2][c] = mixed[2]
        state[3][c] = mixed[3]
    return state
```

1.6 AES Encryption / Decryption Blocks

The individual AES functions were combined and organized into a single encryption and decryption block to simplify the implementation.

`aes_encrypt_block(block16, round_keys)`

```
##### AES ENC Block #####
def aes_encrypt_block(block16, round_keys):
    # block16: 16 bytes plaintext block
    # round_keys: list of 11 round keys (each is 16 bytes)

    # convert input block to AES state (4x4)
    state = bytes_to_state(block16)

    # initial step: AddRoundKey (round 0 key)
    state = add_round_key(state, round_keys[0])

    # rounds 1..9 (full rounds)
    for r in range(1, 10):
        state = sub_bytes(state)      # SubBytes
        state = shift_rows(state)     # ShiftRows
        state = mix_columns(state)     # MixColumns
        state = add_round_key(state, round_keys[r]) # AddRoundKey

    # final round (round 10) - no MixColumns
    state = sub_bytes(state)
    state = shift_rows(state)
    state = add_round_key(state, round_keys[10])

    # convert state back to bytes
    return state_to_bytes(state)
```

`aes_decrypt_block(block16, round_keys)`

```
def aes_decrypt_block(block16, round_keys):
    # block16: 16 bytes ciphertext block
    # round_keys: list of 11 round keys

    state = bytes_to_state(block16)

    # start with last round key
    state = add_round_key(state, round_keys[10])

    # inverse final round steps
    state = inv_shift_rows(state)
    state = inv_sub_bytes(state)

    # rounds 9..1 (inverse full rounds)
    for r in range(9, 0, -1):
        state = add_round_key(state, round_keys[r])
        state = inv_mix_columns(state)
        state = inv_shift_rows(state)
        state = inv_sub_bytes(state)

    # finish with round 0 key
    state = add_round_key(state, round_keys[0])

    return state_to_bytes(state)
```

1.7 CBC Mode Using AES Blocks

`cbc_encrypt(plaintext_bytes, key16)`

In this step, the plaintext message is encrypted using AES-128 in CBC mode. First, the plaintext is padded using PKCS#7 and divided into 16-byte blocks. A fresh random initialization vector (IV) is generated for each message and used to encrypt the first block. Each plaintext block is XORed with the previous ciphertext block before being encrypted using AES. This chaining mechanism ensures that identical plaintext blocks produce different ciphertexts and enhances security.

```
# =====
#           CBC encryption
# =====

def cbc_encrypt(plaintext_bytes, key16):
    # plaintext_bytes: bytes
    # key16: 16 bytes AES key
    # returns: (iv, ciphertext)

    # build round keys from the main key
    words = key_expansion(key16)
    round_keys = words_to_round_keys(words)

    # pad plaintext then split into blocks
    padded = pkcs7_pad(plaintext_bytes)
    blocks = split_blocks(padded)

    # generate random IV
    iv = random_iv()

    ciphertext = b""
    previous = iv

    for block in blocks:
        x = xor_bytes(block, previous)          # Pi XOR previous
        c = aes_encrypt_block(x, round_keys)    # AES encrypt one block
        ciphertext += c                          # chaining
        previous = c

    return iv, ciphertext
```

`cbc_decrypt(iv, ciphertext_bytes, key16)`

During decryption, the ciphertext is first divided into 16-byte blocks. Each block is decrypted using the AES decryption function, and the result is then XORed with the previous ciphertext block (or the IV for the first block) to recover the original plaintext. After processing all blocks, the PKCS#7 padding is removed to obtain the original message. This process correctly reverses the CBC encryption steps and restores the plaintext.

```

# =====
#         CBC decryption
# =====
def cbc_decrypt(iv, ciphertext_bytes, key16):
    # iv: 16 bytes
    # ciphertext_bytes: bytes (multiple of 16)
    # key16: 16 bytes AES key
    # returns: plaintext bytes

    # build round keys
    words = key_expansion(key16)
    round_keys = words_to_round_keys(words)

    blocks = split_blocks(ciphertext_bytes)

    plaintext_padded = b""
    previous = iv

    for block in blocks:
        x = aes_decrypt_block(block, round_keys)      # AES decrypt one block
        p = xor_bytes(x, previous)                   # XOR with previous
        plaintext_padded += p
        previous = block                             # previous ciphertext

    # remove padding
    return pkcs7_unpad(plaintext_padded)

```

1.8 Simple Encryption/decryption Interface

This interface was created to simplify the use of AES-CBC encryption and decryption. Instead of interacting with the internal AES and CBC functions, the encryption process is wrapped into two simple functions that can be easily used for sending and receiving encrypted messages.

encrypt_message(plaintext_str, key16)

This function encrypts a text message and returns the IV and ciphertext in hexadecimal format for easy transmission.

```

def encrypt_message(plaintext_str, key16):
    # plaintext_str: normal text (string)
    # key16: 16-byte AES key
    # returns iv and ciphertext as hex strings |

    plaintext_bytes = plaintext_str.encode("utf-8")
    iv, ct = cbc_encrypt(plaintext_bytes, key16)

    return iv.hex(), ct.hex()

```

✚ `decrypt_message(iv_hex, ct_hex, key16)`

```
def decrypt_message(iv_hex, ct_hex, key16):
    # iv_hex, ct_hex: hex strings
    # key16: 16-byte AES key
    # returns plaintext as string

    iv = bytes.fromhex(iv_hex)
    ct = bytes.fromhex(ct_hex)

    plaintext_bytes = cbc_decrypt(iv, ct, key16)
    return plaintext_bytes.decode("utf-8", errors="replace")
```

2. Phase II: Key Exchange and Authentication

This phase focuses on solving the key distribution problem and ensuring authentication between communicating parties using public-key cryptography. In this phase, RSA-128 digital signatures are used to verify identities based on my student ID, while the Diffie–Hellman key exchange protocol is employed to securely establish a shared session key without transmitting it directly. The derived session key replaces the static key used in Phase I and provides Perfect Forward Secrecy, ensuring that past communications remain secure even if long-term keys are compromised.

2.1 RSA Key Generation, Signing, and Verification

egcd(a, b)

This function implements the Extended Euclidean Algorithm to compute the greatest common divisor of two integers. It also returns coefficients used to calculate the modular inverse in RSA.

```
# =====  
# RSA part (key generation, signing, verifying)  
# =====  
  
# Extended Euclidean Algorithm: to find modular inverse  
def egcd(a, b):  
    if b == 0:  
        return a, 1, 0  
    g, x1, y1 = egcd(b, a % b)  
    return g, y1, x1 - (a // b) * y1
```

modinv(a, m)

This function computes the modular inverse of a modulo m using the Extended Euclidean Algorithm. This function is essential for calculating the RSA private key d.

```
# find inverse of a modulo m: to compute RSA private key d  
def modinv(a, m):  
    result = egcd(a, m)  
    g = result[0]  
    x = result[1]  
    if g != 1:  
        raise ValueError("No modular inverse")  
    return x % m
```

`is_probable_prime(n, rounds)`

This function checks whether a number is prime using the Miller–Rabin test. It enables fast and reliable testing for large numbers used in RSA.

```
# check if number is prime (Miller-Rabin)
def is_probable_prime(n, rounds=8):
    if n < 2:
        return False

    small_primes = [2,3,5,7,11,13,17,19,23,29]
    for p in small_primes:
        if n % p == 0:
            return n == p

    d = n - 1
    s = 0
    while d % 2 == 0:
        d //= 2
        s += 1

    for _ in range(rounds):
        a = secrets.randbelow(n - 3) + 2
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
            continue
        for __ in range(s - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False
    return True
```

`gen_prime(bits)`

This function generates a random prime number with a specified bit length, and ensures the generated prime is suitable for cryptographic use.

```
# generate a prime number of given bit size
def gen_prime(bits):
    while True:
        x = secrets.randbits(bits)

        # make it odd
        if x % 2 == 0:
            x = x + 1

        # ensure correct size
        min_val = 1 << (bits - 1)
        if x < min_val:
            x = x + min_val

        if is_probable_prime(x):
            return x
```

RSAKeyPair

This class defined to store the RSA modulus and key components (n, e, d).

```
# store RSA keys
class RSAKeyPair:
    def __init__(self, n, e, d):
        self.n = n
        self.e = e
        self.d = d
```

rsa_generate_128()

This function generates an RSA-128 key pair using two randomly generated 64-bit prime numbers. It computes the public key (n, e) and the private key (n, d).

```
# generate RSA-128 key pair
def rsa_generate_128():
    p = gen_prime(64)
    q = gen_prime(64)
    while p == q:
        q = gen_prime(64)

    n = p * q
    phi = (p - 1) * (q - 1)

    e = 65537
    if egcd(e, phi)[0] != 1:
        raise ValueError("Bad e")

    d = modinv(e, phi)
    return RSAKeyPair(n, e, d)
```

sha256(data)

This function computes the SHA-256 hash of the input data. Hashing ensures message integrity before signing or verification.

```
# SHA-256 hash function
def sha256(data):
    hash_object = hashlib.sha256(data)
    hash_value = hash_object.digest()
    return hash_value
```


`rsa_sign(message, private_key)`

This function creates a digital signature by encrypting the message hash with the RSA private key.

```
# RSA signing
def rsa_sign(message, private_key):
    hash_bytes = sha256(message)
    h = int.from_bytes(hash_bytes, "big")
    h = h % private_key.n
    signature = pow(h, private_key.d, private_key.n)
    return signature
```

`rsa_verify(message, signature, n, e)`

This function verifies the digital signature using the sender's RSA public key. It confirms that the message was not changed and was signed by the legitimate sender.

```
# RSA verification
def rsa_verify(message, signature, n, e):
    hash_bytes = sha256(message)
    h = int.from_bytes(hash_bytes, "big")
    h = h % n
    recovered_hash = pow(signature, e, n)
    return recovered_hash == h
```

2.2 Diffie–Hellman Key Exchange and Session Key Derivation

🚦 DH_P & DH_G

```
# =====  
#           Diffie–Hellman part  
# =====  
  
# public DH parameters (agreed by both sides)  
DH_P = int(  
    "FFFFFFFFFFFFFFFFC90FDAA22168C234C4C6628B80DC1CD1"  
    "29024E088A67CC74020BBEA63B139B22514A08798E3404DD"  
    "EF9519B3CD3A431B302B0A6DF25F14374FE1356D6D51C245"  
    "E485B576625E7EC6F44C42E9A637ED6B0BFF5CB6F406B7ED"  
    "EE386BFB5A899FA5AE9F24117C4B1FE649286651ECE45B3D"  
    "C2007CB8A163BF0598DA48361C55D39A69163FA8FD24CF5F"  
    "83655D23DCA3AD961C62F356208552BB9ED529077096966D"  
    "670C354E4ABC9804F1746C08CA18217C32905E462E36CE3B"  
    "E39E772C180E86039B2783A2EC07A28FB5C55DF06F4C52C9"  
    "DE2BCBF6955817183995497CEA956AE515D2261898FA0510"  
    "15728E5A8ACAA68FFFFFFFFFFFFFFFF",  
    16  
)  
  
# generator  
DH_G = 2
```

🚦 dh_private()

This function generates a random private value for the Diffie–Hellman protocol.

This value remains secret and is unique for each session.

```
# generate random DH private value  
def dh_private():  
    random_value = secrets.randbelow(DH_P - 2)  
    private_value = random_value + 2  
    return private_value
```

🚦 dh_public(x)

This function computes the Diffie–Hellman public value using the private value x.

The result is safely shared with the other communicating party.

```
# compute DH public value  
def dh_public(x):  
    return pow(DH_G, x, DH_P)
```

`dh_shared(their_public, my_private)`

This function computes the shared secret using the received public value and the local private value. Both parties independently obtain the same shared secret without transmitting it.

```
# compute DH shared secret
def dh_shared(their_public, my_private):
    return pow(their_public, my_private, DH_P)
```

`int_to_bytes(x)`

This function converts an integer into its corresponding byte representation. This conversion is required before applying cryptographic hash functions.

```
# convert integer to bytes
def int_to_bytes(x):
    bits = x.bit_length()
    bytes_len = (bits + 7) // 8
    if bytes_len == 0:
        bytes_len = 1
    return x.to_bytes(bytes_len, "big")
```

`derive_session_key(shared_secret)`

This function derives a 128-bit session key from the Diffie–Hellman shared secret. The session key is generated as `SHA256(s)[0:16]` and used for secure communication.

```
# derive session key from shared secret: session_Key = SHA256(s)[0:16]
def derive_session_key(shared_secret):
    secret_bytes = int_to_bytes(shared_secret)
    full_hash = sha256(secret_bytes)
    session_key = full_hash[:16]
    return session_key
```

2.3 Signed Diffie–Hellman Message Format

 `signed_data(sender_id, receiver_id, dh_value)`

This function constructs a message that binds the sender ID, receiver ID, and Diffie–Hellman public value into a single string.

```
# =====  
#           Signed message format  
# =====  
  
# bind DH value to my id  
def signed_data(sender_id, receiver_id, dh_value):  
    sender = sender_id  
    receiver = receiver_id  
    dh_str = str(dh_value)  
    message = sender + "|" + receiver + "|" + dh_str  
    return message.encode()
```

3. Network Capture

3.1 Terminal output

Both client and server compute the same DH shared secret, then derive:

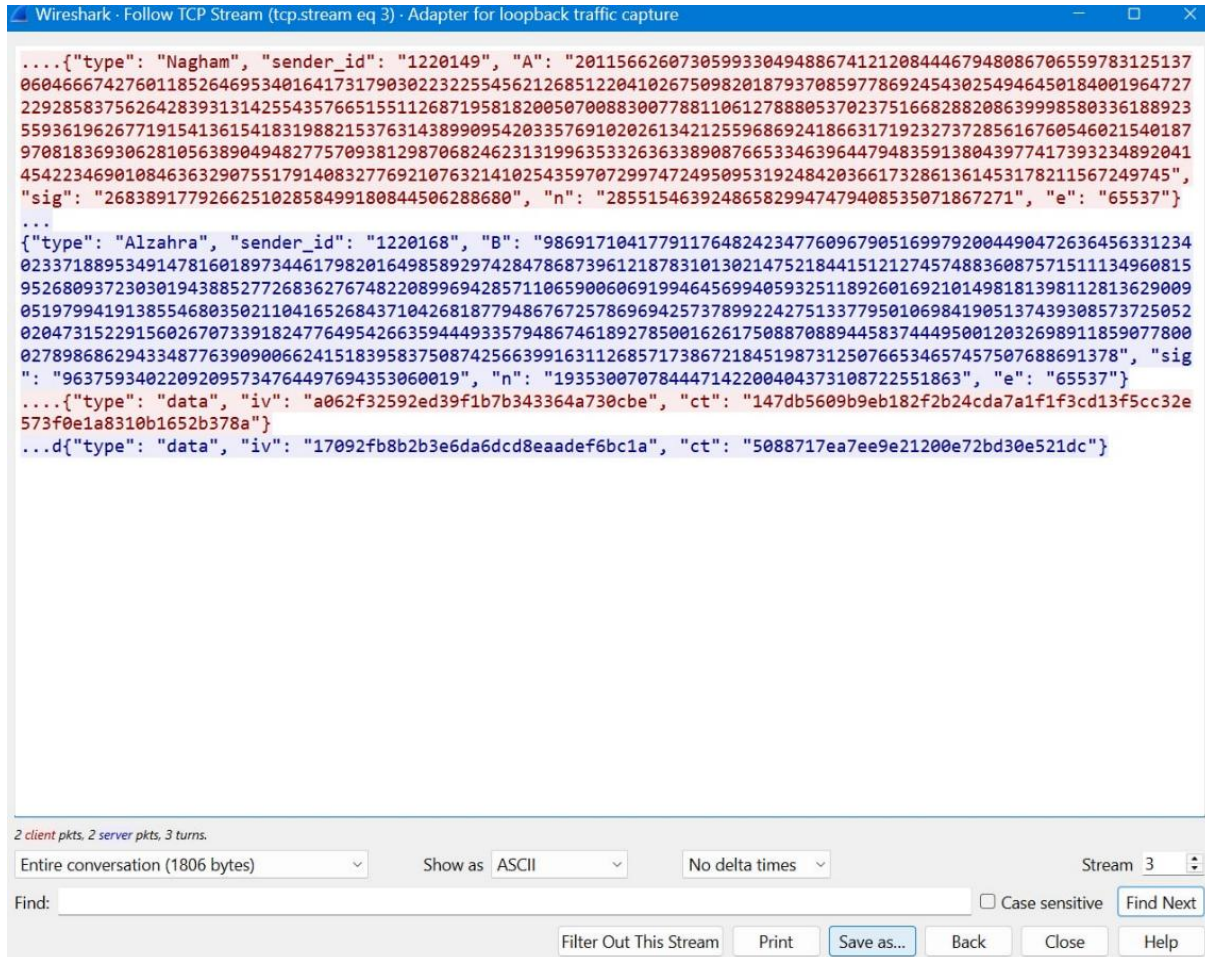
- `Session_Key = SHA256(s)[0:16]` using `derive_session_key()`

```
C:\Users\HP\OneDrive\Attachments\Desktop\Crypto HW2>python server.py
Server listening on 127.0.0.1:5055
Client connected: ('127.0.0.1', 57802)
Alice signature valid? True
Bob session key: 379a8354d5b914ad2a5304c1772b9b10
Decrypted from client: hello over network (AES-CBC)
```

```
C:\Users\HP\OneDrive\Attachments\Desktop\Crypto HW2>python client.py
Connected to server
Bob signature valid? True
Alice session key: 379a8354d5b914ad2a5304c1772b9b10
Decrypted reply: received ok
```

The output confirms that Phase II handshake was completed correctly. First, the server starts listening and accepts the client connection, proving the TCP channel is established. Then each side verifies the other party's RSA-128 signature over the signed Diffie–Hellman value (shown by “signature valid? True”), which provides authentication and prevents impersonation. After verification, both parties compute the same Diffie–Hellman shared secret and derive an identical 128-bit session key using `Session_Key = SHA256(s)[0:16]`, which is why the printed session key matches on both terminals. Finally, the client encrypts a message using AES-CBC with the derived session key, and the server successfully decrypts it (“Decrypted from client...”), then sends an encrypted reply that the client decrypts (“Decrypted reply: received ok”), proving confidentiality and correctness of the session-key-based encrypted communication.

3.2 Wireshark output



The screenshot shows the Wireshark interface with a TCP stream selected. The packet list on the left shows three packets. The packet details pane on the right shows the selected packet (packet 3) with the following JSON data:

```
....{"type": "Nagham", "sender_id": "1220149", "A": "20115662607305993304948867412120844467948086706559783125137060466674276011852646953401641731790302232255456212685122041026750982018793708597786924543025494645018400196472722928583756264283931314255435766515511268719581820050700883007788110612788805370237516682882086399985803361889235593619626771915413615418319882153763143899095420335769102026134212559686924186631719232737285616760546021540187970818369306281056389049482775709381298706824623131996353326363389087665334639644794835913804397741739323489204145422346901084636329075517914083277692107632141025435970729974724950953192484203661732861361453178211567249745", "sig": "268389177926625102858499180844506288680", "n": "285515463924865829947479408535071867271", "e": "65537"}
...
{"type": "Alzahra", "sender_id": "1220168", "B": "986917104177911764824234776096790516997920044904726364563312340233718895349147816018973446179820164985892974284786873961218783101302147521844151212745748836087571511134960815952680937230301943885277268362767482208996942857110659006069199464569940593251189260169210149818139811281362900905197994191385546803502110416526843710426818779486767257869694257378992242751337795010698419051374393085737250520204731522915602670733918247764954266359444933579486746189278500162617508870889445837444950012032698911859077800027898686294334877639090066241518395837508742566399163112685717386721845198731250766534657457507688691378", "sig": "96375934022092095734764497694353060019", "n": "193530070784447142200404373108722551863", "e": "65537"}
....{"type": "data", "iv": "a062f32592ed39f1b7b343364a730cbe", "ct": "147db5609b9eb182f2b24cda7a1f1f3cd13f5cc32e573f0e1a8310b1652b378a"}
...d{"type": "data", "iv": "17092fb8b2b3e6da6dcd8eaadef6bc1a", "ct": "5088717ea7ee9e21200e72bd30e521dc"}
```

The packet list on the left shows three packets:

- 2 client pkts, 2 server pkts, 3 turns.
- Entire conversation (1806 bytes)
- Show as ASCII
- No delta times
- Stream 3
- Find:
- Case sensitive ☐
- Find Next
- Filter Out This Stream
- Print
- Save as...
- Back
- Close
- Help

The captured TCP stream shows the complete Phase II handshake, including the exchange of Diffie–Hellman public values and RSA signatures for authentication. After successful key agreement, encrypted messages are exchanged using the derived session key. The transmitted data contains only initialization vectors (IVs) and ciphertexts, and no plaintext information is visible in the network capture.